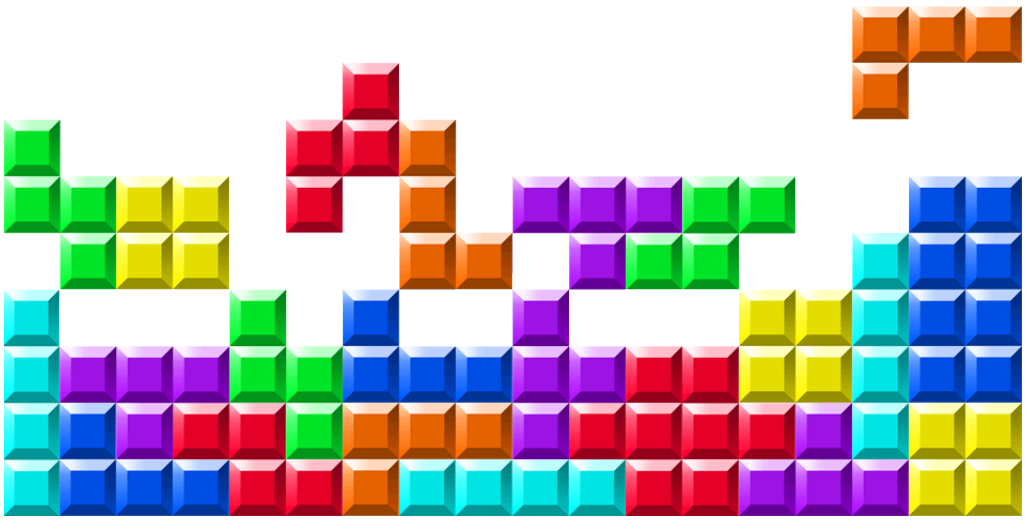
LINFO2275
Second project

Group 2:

Gavin Hope 09022500 – **SINF**
Jakob Thibodeau 12752500 – **DATI**
Lore Lernoux 77562000 – **MAP**



1 Introduction

Reinforcement learning is a useful framework for problems where an agent has to take decisions step by step. The agent does not know in advance what the best action is, but it learns by interacting with the environment and by receiving rewards. This is why games are often used to test reinforcement learning algorithms, since every action can influence not only the current state, but also the future states of the game.

In the first part of the project, we studied Markov decision processes and value iteration on the Snakes and Ladders game. In this second part, we wanted to investigate a more complex application, where exact methods such as value iteration are not really practical anymore. For this reason, we chose to work on Tetris. Tetris is an interesting game for reinforcement learning because the player must decide where to place each piece depending on the current board. Some decisions can look good in the short term, for example clearing one line, but can also create holes or make the board harder to play later. Therefore, the agent has to learn a strategy that takes into account both immediate and future rewards.

The goal of this project is to study how Deep Q-Learning can be used to train an agent to play a simplified version of Tetris. Since the number of possible board configurations is very large, it is not possible to use a classical Q-table with one value for each state-action pair. Instead, we use neural networks to approximate the Q-values. We compare two different ways of representing the state. The first one uses the full board as a binary grid and is trained with a convolutional neural network. The second one uses a smaller set of features, such as the number of holes, the height of the board, the bumpiness, and the number of cleared lines.

We also investigate the effect of the reward function on the learned behaviour. Indeed, the way rewards are defined can strongly influence what the agent learns. For example, rewarding the agent for clearing lines may lead to a different strategy than rewarding it for surviving longer and placing more blocks. In our experiments, we compare several reward schemes and evaluate the trained models according to different criteria, such as the number of blocks placed or the number of lines cleared.

The main questions we want to answer are:

- Can a Deep Q-Learning agent learn a reasonable strategy for Tetris?
- Does the choice of state representation have an important impact on the performance?
- How does the reward function change the behaviour of the agent?
- Which model performs best in terms of survival time and number of lines cleared?

The rest of the report is organised as follows. First, we introduce the theoretical background of Q-learning and Deep Q-Learning. Then, we describe the Tetris environment, the state and action spaces, and the reward functions used during training. Finally, we present the experimental results and compare the different trained models.

2 Theoretical Background

2.1 Q-learning

Q-learning is a model-free reinforcement learning algorithm for learning optimal behaviour through interaction with an environment. It is model-free because it does not require a model of the environment, such as the transition probabilities or reward function. Instead, the agent learns from experience, by trying actions and observing the obtained rewards. In the standard formulation, the environment is modelled as a Markov decision process (MDP) defined by:

- a set of states $\mathcal{S}$,
- a set of actions $\mathcal{A}$,
- a transition probability distribution $p(s', r \mid s, a)$ that gives the probability of transitioning to state s' and receiving reward r after taking action a in state s,
- a discount factor $\gamma \in [0, 1]$ that determines the importance of future rewards.

At time t, the agent observes a state S_t, selects an action A_t, receives a reward R_{t+1}, and moves to a new state S_{t+1}. The objective is to learn a policy that maximises the expected return, where the return is the discounted sum of future rewards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

The agent learns by estimating action-values, also called Q-values. A Q-value represents how good it is to take a given action in a given state. More precisely, for a policy π, the action-value function is defined as

$$q_\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid S_t = s, A_t = a \right].$$

The goal of Q-learning is to learn the optimal action-value function

$$q_*(s, a) = \max_\pi q_\pi(s, a).$$

The optimal Q-values satisfy the Bellman optimality equation:

$$q_*(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right].$$

In the tabular version of Q-learning, the agent stores one value $Q(s, a)$ for each state-action pair. After observing a transition

$$(S_t, A_t, R_{t+1}, S_{t+1}),$$

the Q-value is updated as follows [SB18]:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right].$$

A common way to select actions during learning is the ε-greedy strategy. With probability $1 - \varepsilon$, the agent chooses the action with the highest current Q-value, while with probability ε, it chooses a random action [Gee25b].

2.2 Deep Q-Learning

In classical Q-learning, the agent stores the values $Q(s, a)$ in a table. This works when the number of states and actions is small, but it becomes impossible for larger problems. For example, in Tetris, the number of possible board configurations is very large, so a Q-table would be too big and most states would almost never be visited during training.

Deep Q-learning solves this problem by replacing the Q-table with a neural network. Instead of storing one value for each state-action pair, the algorithm learns a function [Gee25a]

$$Q(s, a; \theta),$$

where θ are the parameters of the neural network. The network receives a representation of the state as input and predicts Q-values. These predicted values are then used to choose the action, usually with an ε-greedy strategy during training.

The learning target is still based on the same idea as in Q-learning. For a transition

$$(S_t, A_t, R_{t+1}, S_{t+1}),$$

the target value is

$$y_t = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a'; \theta).$$

The neural network is trained to make its prediction $Q(S_t, A_t; \theta)$ closer to this target. This can be done by minimizing the squared error

$$\mathcal{L}(\theta) = \left(y_t - Q(S_t, A_t; \theta)\right)^2.$$

In our project, Deep Q-learning is useful because Tetris has too many possible states for tabular Q-learning. The neural network allows the agent to choose a right action even if he did not see the exact same board configuration before, by generalizing from similar situations.

3 Implementation

In this project, the Tetris environment is modeled as a Markov Decision Process (MDP) and solved using Deep Q-Networks (DQN). The implementation bridges the gap between raw game states and the reinforcement learning framework. The game and implementation of DQN are extensions of the work of Viet Nguyen [Ngu23].

Our first attempt was to use Proximal Policy Optimization (PPO), which is a policy-based algorithm. However, we found that a policy-based method was not well-suited for Tetris, as the number of actions leading to the same outcome is very large. Indeed, the way pieces arrive at their final position does not really matter, so the action *move left* is not interesting in itself, as we only care about the final position. This is why we switched to a value-based method, which focuses on learning the value of states and actions rather than directly learning a policy [Eyc23].

3.1 Environment

Tetris is a game where blocs of differents shapes and sizes appear at the top of a grid board and fall with time. The goal is to place as many blocks as possible without filling in the whole grid. If an entire row is filled when placing a block, that entire row disappears, and the player scores points. The highest number of rows that can be removed at once is 4, and doing so is called a *Tetris*.

Several different reward functions were explored for training the Q-Learning models. For the sake of thoroughness, up to 300,000 epochs per reward scheme were tested, and results for each model trained on 50,000, 100,000, 150,000, 200,000, 250,000, and 300,000 are shown in Figures 1, 2, and 3 in Section 4.2.

The three different reward schemes tested were:

- **Lines Flat:** The model receives +1.0 per line cleared, -2.0 for losing the game.
- **Lines Bonus:** Simulating original Tetris scoring behavior, the model receives +1.0 for clearing a single line, +3.0 for clearing two lines at the same time, +5.0 for clearing three lines at the same time, and +8.0 for clearing four lines at the same time. It also receives the same penalty of -2.0 for losing the game.

- **Per Block:** This model receives +1.0 per block placed and -2.0 for losing the game.

A more advanced and complicated reward function was tested. This function used the lines bonus, but also included a penalty for creating overhands in the board. Any time that an overhang was created, a penalty of -1.0 was added. Further, after every block placed, the model would receive a reward relative to how low the current stack was. The motivation for this is readily apparent, as it further incentivizes the model to play "properly". In testing, however, the model consistently failed to clear more than a single line even after 300,000 training epochs. This is likely due to a number of factors. This is likely due to a combination of the disincentives quickly heavily outweighing the incentives, combined with the Q-learning model having no actual visual representation of the board, as outlined in the following subsection. This reward function is thus excluded from later experimentation.

3.2 Action Space

Unlike traditional Tetris where moving pieces is possible while they fall, the agents have access to a *simplified action space*. An action a is defined as the tuple (x, r), where x is the column where the piece should be dropped and r is the number of rotation of the piece. Since the grid is 10 units wide and a piece can be rotated at most 4 times, the action space contains 40 different actions.

3.3 Deep Q-Learning (DQN)

In complex environments like Tetris, the state space $\mathcal{S}$ is too large and therefore a traditional Q-value table would be very sparse and hard to learn from. Instead, Deep Q-Learning approximates $q_*(s, a)$ using a neural network $Q(s, a; \theta)$, where θ represents the weights of the network.

In this implementation, two distinct neural architectures are used: Neural Networks and Convolutional Neural Networks.

- **Convolutional Neural Network (CNN):** This architecture is used to process the 20×10 binary grid as an image. It utilizes three convolutional layers to extract spatial features and geometric patterns from the Tetris board, followed by dense layers to output the predicted Q-value.
- **Neural Network:** This model consists of three fully connected (linear) layers. It is designed to process a 4-dimensional *smart* feature state vector [`lines, holes, bumpiness, height`] (see 3.4). The layers use ReLU activation functions to introduce non-linearity, mapping the four features to a single scalar Q-value.

3.4 State Representation (S)

The state space is handled by the `get_state_properties` method in `tetris.py`, offering two distinct representations based on the chosen architecture.

The observation fed to the CNN is a 20×10 binary array where 1 represents the presence of a block, and 0 represents an empty space.

For the standard NN, the state representation is simplified to 4 heuristic features:

- **Lines Cleared**: number of lines cleared as a result of the given action.
- **Height**: Sum of the height of all columns.
- **Bumpiness**: Sum of the absolute differences in height between adjacent columns.
- **Holes**: Number of empty spaces with a filled space somewhere above it.

4 Experiments and Results

4.1 Comparison of state representations

To begin with, both the raw board CNN and heuristic NN models were given the same reward function for training.

The first obvious difference between the 2 was the amount of epochs needed for the models to understand the game.

JAKOB CREATE GRAPHICS TO COMPARE MODELS

In terms of behavior, the raw CNN model starts every game by placing blocks at random with no evident strategy. Only when it gets close to losing does it start placing blocks strategically, prolonging the game further.

The model fails to understand that the placement of the earlier pieces affects the score. Furthermore, it cannot take advantage of the exponential scoring of clearing multiple lines and does not seem to aim for Tetrises.

On the other hand, the heuristic NN model is very efficient in its block placement from the very beginning. Frequently, it leaves an entire column empty so that it can fit a line piece to score a Tetris.

However, the model sometimes over-focuses on achieving Tetrises, and ends up losing the game before getting the line piece in.

4.2 Variations in given Rewards

Each epoch checkpoint was evaluated over 100 games with a maximum of 100,000 piece placements per game to prevent overly long games. During testing, the model always picked the action with the highest predicted Q-value with no random exploration.

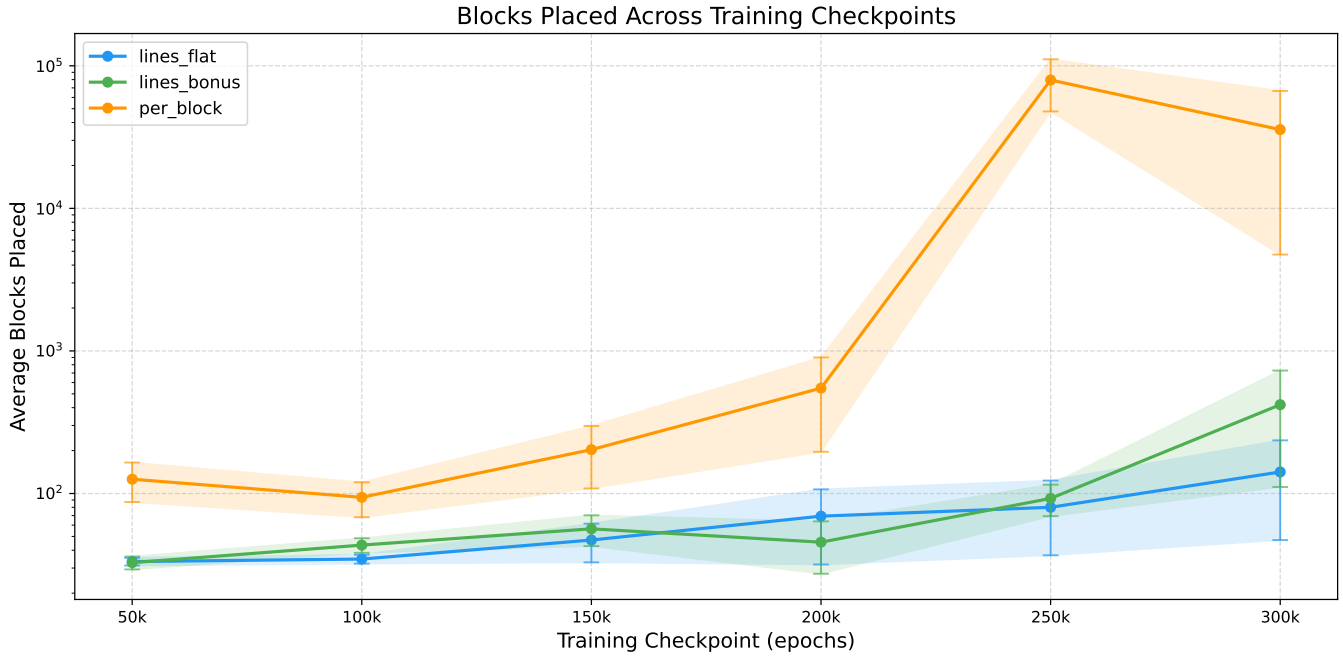


Figure 1: The average number of blocks placed by each reward model per game over 100 games at different training epochs.

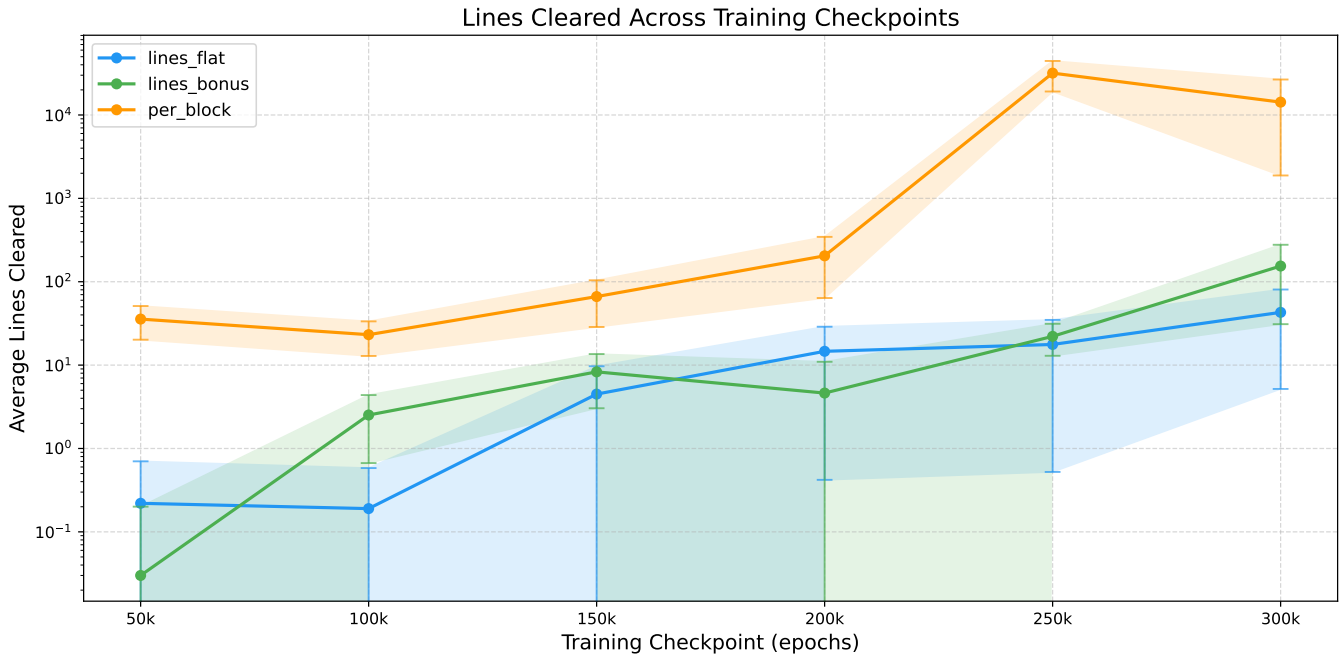


Figure 2: The average number of lines cleared by each reward model per game over 100 games at different training epochs.

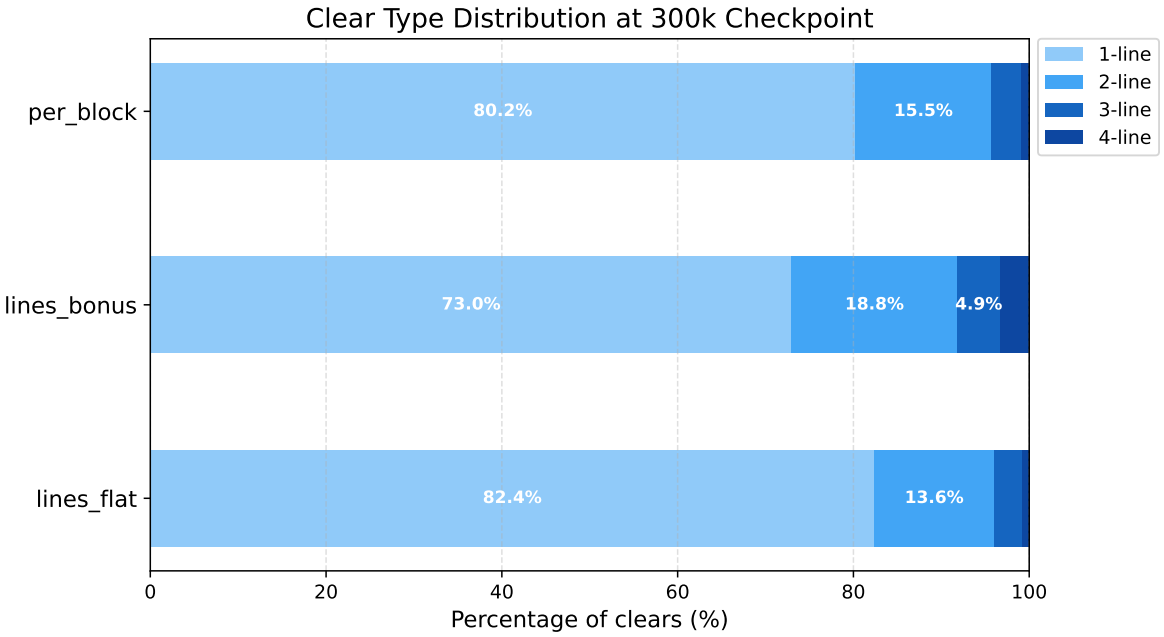


Figure 3: The distribution of number of lines cleared at same time by each reward model at different training epochs over 100 games.

As shown, the Per Block reward scheme produced by far the most resilient model from both the perspective of total blocks placed and lines cleared. The Flat Lines reward scheme seemingly produced the least consistent model, with many games resulting in little to no successful lines cleared. As shown in Figure 3, the Lines Bonus scheme produced a model that favored multiple line clears at the same time, whereas Per Block displayed a stronger preference to clearing a single line at a time. This stands to reason, as the Lines Bonus was actively incentivized to clear as many lines at once as possible, whereas Per Block was more focused on surviving as long as possible.

One somewhat surprising result visible in Figures 1 and 2 is that some models actually perform worse at certain later checkpoints than earlier ones. This is seemingly a result of the replay buffer having a fixed size, so older experience gets overwritten over time. Therefore, if the model goes through a particularly bad patch, the buffer fills up with low-quality gameplay, the network trains on that, and performance drops further. This effect is made worse by the fact that by the time the model has reached 300,000 epochs, the value of epsilon is nearly zero, meaning there is essentially no random exploration to help the model recover from whatever suboptimal strategy it has settled into.

5 Conclusion

In this project, we implemented Deep Q-Learning to train agents to play a simplified version of Tetris. We compared two different state representations: first, a heuristic feature representation processed by a standard neural network, and then a convolutional neural network processing a binary board representation. We also investigated the effect of different reward functions on the behaviour of the agents.

Our results show that the choice of state representation has a significant impact on the performance. The heuristic feature representation allowed the agent to learn a reasonable strategy more quickly, while the binary board representation required more training and did not perform as well. This suggests that providing a more compact state representation can help the learning process.

Regarding the reward functions, we found that rewarding the agent for placing blocks (Per Block) led to better performance in terms of survival time and number of lines cleared, compared to rewarding only for clearing lines (Lines Flat and Lines Bonus). This indicates that focusing only on clearing lines is not sufficient for the agent to survive longer, and place more blocks can lead to a more efficient strategy.

Overall, our experiments demonstrate that Deep Q-Learning can be used to train agents to play Tetris, but the choice of state representation and reward function is crucial for achieving good performance.

References

[Eyc23] Mustafa Eyceoz. *Tetris via Reinforcement Learning: A Lesson in Simplicity*. Accessed: 2026-05-15. 2023. URL: https://github.com/Maxusmusti/tetris-ppo/.

[Gee25a] GeeksforGeeks. *Deep Q-Learning in Reinforcement Learning*. Accessed: 2026-05-12. 2025. URL: https://www.geeksforgeeks.org/deep-learning/deep-q-learning/.

[Gee25b] GeeksforGeeks. *Q-Learning in Reinforcement Learning*. Accessed: 2026-05-07. 2025. URL: https://www.geeksforgeeks.org/machine-learning/q-learning-in-python/.

[Ngu23] Viet Nguyen. *Tetris-deep-Q-learning-pytorch*. Accessed: 2026-05-03. 2023. URL: https://github.com/vietnh1009/Tetris-deep-Q-learning-pytorch.

[SB18] Richard S. Sutton and Andrew G. Barto. "Reinforcement Learning: An Introduction". In: 2nd ed. Cambridge, MA: MIT Press, 2018, pp. 157–159.